



Observações Gerais

- É vedado o uso de ferramentas de inteligência artificial ou modelos de linguagem (LLMs).
- A constatação de utilização de tais recursos implicará na atribuição de nota **zero**.
- O discente poderá ser chamado para apresentar e justificar o código desenvolvido.
- A reprodução parcial ou integral do trabalho de outro colega resultará igualmente em nota **zero**.
- Durante a aula, algumas resoluções dos alunos serão selecionadas por sorteio para apresentação e debate.

Retrocedendo

Nos sistemas operacionais, a fila de processos é usada para gerenciar a execução de programas pela CPU. Cada processo entra na fila quando está pronto para ser executado. A CPU retira o primeiro processo da fila (FIFO – First In, First Out) e o executa por um tempo fixo chamado **quantum**. Se o processo não terminar dentro do quantum, ele volta para o fim da fila e aguarda sua próxima vez. A Figura 1 ilustra o sequenciamento das tarefas usando esse algoritmo.

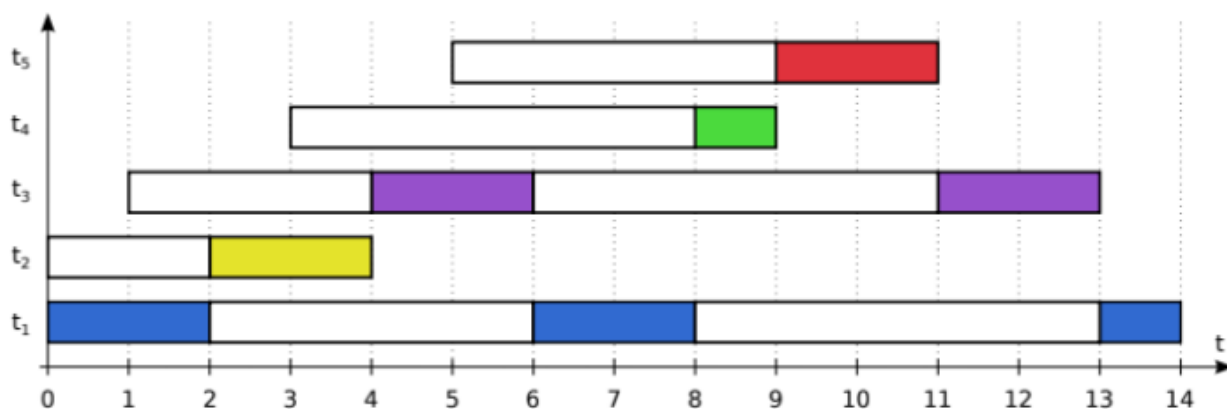


Figura 1: Escalonamento de processos pela sistema operacional.

Nesta atividade, você vai implementar um simulador de escalonamento de processos usando fila em Java.
Requisitos

1. Crie uma classe Processo com os seguintes atributos:
 - nome (ex.: "P1", "P2")
 - instrucoesRestantes (quantidade de instruções que o processo precisa executar)
 - tempoChegada (momento em que o processo entra na fila)
 - status (PRONTO, EXECUTANDO, TERMINADO)
2. Implemente um escalonador que:
 - Use uma fila para gerenciar os processos.
 - Considere um quantum fixo de 2 instruções por rodada.

- A cada tempo, verifique se novos processos chegaram e insira-os na fila.
- Execute o processo da frente da fila por até 2 instruções.
- Se o processo terminar, atualize o status para TERMINADO.
- Se não terminar, volte o processo para o fim da fila com status PRONTO.

3. Considere 5 processos com os seguintes tempos de chegada:

- P1: 7 instruções, chega no tempo 0
- P2: 4 instruções, chega no tempo 0
- P3: 5 instruções, chega no tempo 1
- P4: 6 instruções, chega no tempo 2
- P5: 3 instruções, chega no tempo 4

Saída esperada

1. O programa deve imprimir no console:

- Quando cada processo chega e entra na fila.
- Quando cada processo é executado (quantas instruções foram rodadas e quantas restam).
- Quando cada processo termina.
- Mensagens indicando se a fila está vazia em algum tempo.

Exemplo simplificado:

```
Tempo 0: P1 chegou e entrou na fila.
Tempo 0: P2 chegou e entrou na fila.
P1 executando...
(espera 2 segundos)
P1 executou 2 instruções. Restam: 5

Tempo 1: P3 chegou e entrou na fila.
P2 executando...
(espera 2 segundos)
P2 executou 2 instruções. Restam: 2

Tempo 2: P4 chegou e entrou na fila.
P1 executando...
(espera 2 segundos)
P1 executou 2 instruções. Restam: 3

Tempo 4: P5 chegou e entrou na fila.
P2 executando...
(espera 2 segundos)
P2 executou 2 instruções. Restam: 0
P2 terminou!

...
```

A espera de 2 segundos (representação do quantum) pode ser feita via sleep

```
try {
    System.out.println(nome + " executando...");
    Thread.sleep(2000); // pausa 2 segundos
} catch (InterruptedException e) {
    e.printStackTrace();
}
```

2. Apresente em um gráfico de como ficou os processos em execução semelhante a Figura 1.